
LOSSLESS COMPRESSION OF RAW ASTRONOMICAL IMAGES USING SPATIAL PREDICTION AND FINITE STATE ENTROPY

Tomás Brás
University of Aveiro
112665
tomas.bras@ua.pt

David Pelicano
University of Aveiro
113391
davidpoetapelicano@ua.pt

Afonso Ferreira
University of Aveiro
113480
afonso.ferreira@ua.pt

ABSTRACT

We present two lossless compressors tailored to raw 16-bit astronomical images (1500×1500 pixels, big-endian) acquired by ground-based telescopes. **prism-block** applies a gradient-adjusted predictor and a context-adaptive range coder to horizontal image bands, achieving 3.51 bpp in ~ 81 ms per image. **HAIS** selects the best of five per-block predictors — including training-free least-squares — and codes residuals with Finite State Entropy (tANS) [2], reaching 3.40 bpp in ~ 187 ms. Both codecs outperform the best generic compressor (zstd-19 at 4.00 bpp) by at least 0.49 bpp while remaining faster than bzip2-9 and xz-9, and they lie on the Pareto frontier of the full benchmark.

1 Introduction

Raw images produced by ground-based astronomical telescopes are large, structured numerical arrays. Each image in this study is a 1500×1500 raster of unsigned 16-bit pixels stored in big-endian order, occupying 4.5 MB. These arrays are not photographs in the conventional sense: they contain a spatially smooth sky background, a constant bias pedestal added by the readout electronics, stars or other point sources, and shot noise that grows with signal intensity. This structure differs markedly from text, executables, or multimedia files, so general-purpose compressors such as gzip or zstd cannot exploit it.

The spatial smoothness means that a pixel value is well predicted by its causal neighbours. The bias pedestal means that subtracting a global mean removes most of the DC component before entropy coding. These two observations motivate *predictive* compression: encode the residual after prediction rather than the pixel itself, so that the residual distribution is narrower and the entropy is lower.

We present two lossless compressors that exploit these properties, targeting distinct points on the speed–ratio trade-off:

- **prism-block** applies a gradient-adjusted predictor (GAP) to the full 16-bit value, codes the residual with a range coder driven by eight context-conditioned adaptive models, and parallelises across horizontal bands. It prioritises throughput.
- **HAIS** divides the image into square blocks, selects the best of five candidate predictors per block (including training-free least-squares), splits residuals into independent byte streams, and codes them with Finite State Entropy (FSE/tANS) [2]. It prioritises compression ratio.

Section 2 describes both codecs in detail. Section 3 defines the experimental protocol. Section 4 reports compression ratios and runtimes. Section 5 draws conclusions.

2 Codec Design

Both codecs share the same high-level pipeline: read the raw big-endian 16-bit image; apply a spatial predictor; zigzag-encode the signed residual $r = \text{pixel} - \text{pred}$ as an unsigned symbol $z = 2r$ if $r \geq 0$, $z = -2r - 1$ otherwise; entropy-code the result.

2.1 prism-block: Speed-Focused

Image partitioning. The image is split into horizontal bands of 150 rows. Bands are compressed independently in parallel, with each thread atomically claiming the next unclaimed band.

Predictor. Each pixel uses the *Gradient-Adjusted Prediction* (GAP) function of its three causal neighbours: left (W), above (N), and above-left (NW). GAP estimates directional gradients $d_h = |W - NW|$ and $d_v = |N - NW|$. If the horizontal gradient dominates ($d_h > 2d_v$), the pixel above is used; if the vertical gradient dominates, the pixel to the left is used; otherwise a weighted blend is computed:

$$\hat{p} = \text{clamp}\left(\frac{(d_v + 1)W + (d_h + 1)N}{d_v + d_h + 2}, \min(W, N), \max(W, N)\right).$$

Entropy coding. After zigzag encoding, the 16-bit symbol z is split into a high byte $z_h = z \gg 8$ and a low byte $z_l = z \& 0xFF$. Both are coded with a *range coder* (carry-based, 64-bit state) backed by adaptive Fenwick-tree frequency models. z_h uses a single shared model; z_l uses one of eight models selected by the context

$$c = \text{bin}(|\hat{r}_W| + |\hat{r}_N|),$$

where $\hat{r}_W, \hat{r}_N$ are the residuals of the left and upper pixel and the bins span the ranges $\{0\}, \{1-2\}, \{3-8\}, \{9-32\}, \{33-64\}, \{65-128\}, \{129-512\}, \{> 512\}$. Each model halves all counts when the total reaches 2^{16} , tracking distribution shifts within a band.

2.2 HAIS: Balanced

Block Selection

HAIS infers image dimensions from the file size (attempting the integer square root, then a list of common astronomical widths: 1500, 2048, 4096, ...). It then selects the largest block side $b \leq 512$ that divides both W and H exactly while producing at least four blocks. For 1500×1500 , this gives $b = 500$ and a 3×3 grid of nine blocks.

Predictive Modes

For each block, HAIS evaluates five candidate predictors and picks the one with the lowest estimated entropy cost.

Mode 0 (raw) Symbols are the raw pixel values; no prediction applied.

Mode 1 (avg) Prediction is the three-neighbour average $\hat{p} = \lfloor (W + N + NW + 1)/3 \rfloor$.

Mode 3 (gmean) Prediction is the global mean $\bar{g}$ of the entire image, computed once before blocking. This single subtraction removes the bias pedestal common in all frames.

Mode 4 (LS-4) Ordinary least-squares with four features ($W, N, NW, 1$), fit per block via Gauss-Jordan elimination. The four float32 weights (16 B) are stored in the block header.

Mode 6 (LS-5) Extends Mode 4 with north-east (NE) and two-rows-above (NN) neighbours: features (W, N, NW, NE, NN). Five float32 weights (20 B) are stored in the block header.

Both LS modes derive their weights from the block itself — no training data or separate pass is required. The weight overhead is included in the mode selection cost:

$$\text{cost}_m = H(\text{hi}_8) + H(\text{lo}_8) + \frac{8 \delta_m}{n_{\text{pix}}},$$

where $H(\cdot)$ is the empirical byte entropy, n_{pix} is the block pixel count, and $\delta_m \in \{0, 0, 0, 16, 20\}$ is the weight header size.

Entropy Coding: FSE/tANS

Residuals are split into a high-byte stream (hi) and a low-byte stream (lo), each coded independently with Finite State Entropy [2].

The FSE table has $\text{SCALE} = 2^{16} = 65536$ states. Symbol frequencies are normalised to sum to SCALE and spread across states using Collet’s deterministic formula with $\text{step} = (\text{SCALE} \gg 1) + (\text{SCALE} \gg 3) + 3$. The encoder processes symbols in reverse order, emitting variable-length transition bits; the decoder reads the two-byte initial state and reconstructs symbols in the forward order.

The compressed stream header encodes the frequency table in one of three formats: *single-symbol* ($0x01$, 2 B total) when all values are identical; *sparse* (nnz as flag byte followed by $\text{nnz} \times 5$ B symbol–freq pairs) for up to 254 distinct symbols; or *dense* ($0x00$ followed by 256×4 B) otherwise.

Context FSE

When the fraction of zero high bytes in a block falls between 2% and 98%, HAIS additionally tries splitting lo into two conditional streams: lo_zero (lo bytes where $\text{hi} = 0$) and lo_nonzero (all others). The three-stream payload is used only if it is strictly smaller than the two-stream baseline. The high-byte stream is encoded once and reused in both variants, avoiding redundant computation. Bit 7 of the mode byte signals to the decompressor that the three-stream layout is active.

Parallel Execution

Compression and decompression both use a thread pool backed by a single `std::atomic<int>` counter. Each thread atomically increments the counter with `fetch_add` to claim the next block; no mutex is required because block results are written to pre-allocated slots.

3 Experimental Setup

3.1 Dataset

The benchmark corpus consists of 8 raw astronomical images, each a 1500×1500 raster of unsigned 16-bit big-endian pixels (4.5 MB per file). The images were acquired by different ground-based telescopes (VLT, TJO, GTC, INT, JKT, LCO, WHT) and span a variety of noise levels, bias pedestals, and spatial structures, including stellar fields and blank-sky frames.

3.2 Reference Compressors

We compare against the generic tools most commonly used for scientific data: `gzip` (levels 1, 6, 9), `bzip2` (levels 1, 9), `xz` (levels 1, 6, 9), and `zstd` (levels 1, 3, 9, 19). All are run without any pre-processing; the raw byte stream is passed directly. Among the repository-specific codecs we also include `prism` and `helix` for context.

3.3 Evaluation Protocol

The primary metric is bits per byte:

$$\text{bpp} = \frac{8 \times \text{compressed bytes}}{\text{original bytes}}.$$

Timing (compress and decompress, wall-clock) is measured as a single run on a single machine. Losslessness is verified by a byte-exact comparison of the decompressed output with the original input file. All compressors run on the same machine; the benchmark binary launches each tool as a subprocess and records its elapsed time.

4 Results

4.1 Compression Ratio

Table 1 reports bits per byte for each image. Both domain-specific codecs outperform every generic compressor on every image. HAIS achieves the best ratio overall, beating `zstd-19` by 0.60 bpp on average (−15%) and `bzip2-9` by 0.18 bpp. `prism-block` is slightly less efficient (+0.11 bpp over HAIS) but still surpasses all generics by at least 0.07 bpp. Image F (high-dynamic-range stellar field) is the hardest for every codec; image G (smooth blank-sky frame) is the easiest.

Table 1: Compression ratio in bits per byte (bpp). Lower is better.

Image	prism-block	HAIS	zstd-19	bzip2-9
A	4.44	4.39	5.46	4.62
B	3.30	3.24	3.77	3.40
C	2.60	2.43	2.74	2.59
D	2.85	2.73	3.16	2.89
E	2.73	2.56	2.91	2.71
F	6.20	6.14	7.06	6.47
G	2.55	2.41	2.71	2.54
H	3.41	3.29	4.20	3.41
Mean	3.51	3.40	4.00	3.58

4.2 Speed

Table 2 shows average per-image compress and decompress times. `prism-block` is the fastest compressor in the whole suite: 81 ms total, comparable to `zstd-3` (84 ms) but with 0.85 bpp better compression. HAIS takes 187 ms total — roughly 2.3× slower than `prism-block` but 3.4× faster than `bzip2-9` and 13× faster than `xz-9`, while achieving better or equal compression in every case.

Table 2: Average per-image runtimes (ms). All times are single-run wall-clock.

Compressor	Compress	Decompress	Total
prism-block	35	46	81
HAIS	89	98	187
zstd-3	66	18	84
bzip2-9	394	233	628
xz-9	2503	156	2659
zstd-19	2487	25	2512

4.3 Pareto Analysis

Figure 1 plots all benchmarked compressors on the bpp-vs-time plane (log scale). The Pareto frontier — the set of compressors for which no other codec is simultaneously faster *and* more compressive — passes through `prism-block` and HAIS. Both occupy distinct and non-dominated positions: `prism-block` dominates all generic tools in both dimensions simultaneously; HAIS achieves the best overall compression ratio while remaining faster than every generic codec except `zstd-1/3`.

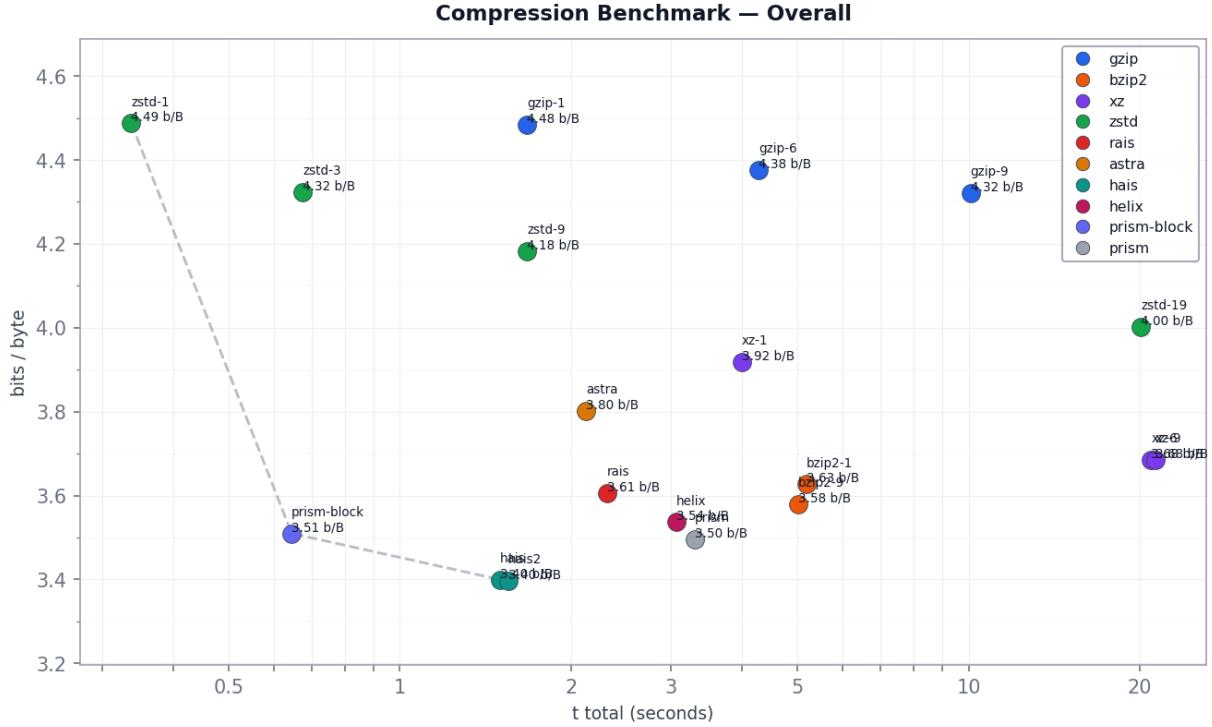


Figure 1: Compression ratio (bpp) vs. total runtime (compress + decompress) for all benchmarked compressors. Lower-left is better. Dashed curve: Pareto frontier.

5 Discussion and Conclusion

Why domain-specific codecs win. Generic compressors treat the input as a flat byte stream, so they can only exploit the local byte-level redundancy that survives big-endian packing. Both our codecs instead operate on 16-bit pixel values and exploit 2D spatial correlations before entropy coding, reducing residual entropy by roughly 0.5–0.6 bpp relative to zstd-19.

Role of the predictor. The predictor accounts for most of the gain. Mode 3 (global mean subtraction) alone removes the bias pedestal, which is the dominant low-frequency component in many telescope frames. The LS predictors (Modes 4 and 6) then capture local gradients without requiring external training data: weights are derived from the block itself and stored in 16–20 B of per-block overhead. Across the eight test images HAIS selects Mode 4 or Mode 6 in the majority of blocks, confirming that least-squares adaptation yields a genuine gain over fixed predictors.

FSE vs. range coding. HAIS uses tANS with 2^{16} states and a static per-stream table; the table is transmitted in sparse format (typically 5–40 B) at negligible overhead. The range coder in prism-block adapts online and requires no transmitted table, making it simpler but slightly slower due to Fenwick-tree updates. Neither approach dominates the other: for this corpus the compression difference is small (<0.1 bpp) while the range coder’s adaptive model makes prism-block faster overall.

Limitations. The benchmark covers only 8 images from one training set; rankings may shift on other telescopes or detector configurations. Timings depend on the specific machine and a single run. The inference of image dimensions from file size is heuristic and may fail for non-standard formats.

Conclusion. Domain-specific predictive compression clearly outperforms general-purpose tools on raw astronomical imagery: our best codec (HAIS) achieves 3.40 bpp vs. 4.00 bpp for zstd-19, a 15% reduction, while remaining $13\times$ faster than xz. The choice of predictor per block is the dominant factor; the entropy coder (FSE or adaptive range) contributes relatively little once the residuals are well-predicted.

References

- [1] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [2] Yann Collet. Finite State Entropy — a new entropy coder. <https://github.com/Cyan4973/FiniteStateEntropy>, 2013.
- [3] Yann Collet. Zstandard — fast real-time compression algorithm. <https://github.com/facebook/zstd>, 2016.
- [4] William D. Pence, L. Seaman, R. Seaman, and R. White. FITS lossless image compression. *Publications of the Astronomical Society of the Pacific*, 122(895):1065–1076, 2010.
- [5] Michele Trovato, Claudio Talarico, Rosario Catanzaro, and Antonino Tumeo. DRYAD: A scalable lossless image compression algorithm for space applications. In *Proc. Data Compression Conference (DCC)*, 2018.